

SAS2Py

User Manual

SAS and SAS-callable SUDAAN translation to R and Python

Current feature guide
September 11, 2026

Contents

1	Purpose
2	Access and Sign-In
3	Dashboard
4	Studio Conversion Workflow
5	SAS Analysis
6	Input Files
7	Automatic Run and Repair
8	Running Generated Code
9	Editing and Applying Enhancements
10	Reviews
11	History and Projects
12	Settings
13	Common Validation and Error Cases
14	Recommended Review Workflow
15	Administrator Notes

The PDF and in-app Documentation page describe the same current workflow.

1. Purpose

SAS2Py converts SAS and SAS-callable SUDAAN programs into R or Python. It keeps the original source and generated code together and stores enhancement, execution, and review history for each saved conversion.

SAS2Py assists code modernization and review. Generated statistical code still requires subject-matter review, especially when an exact SAS or SUDAAN method is not available in the target language.

The application includes:

- Dashboard
- Studio
- History and projects
- Documentation
- Settings

2. Access and Sign-In

Authentication is required for the main workspace. The current sign-in flow uses internal email and password accounts stored by the application.

Signing in provides access to saved conversions, execution records, reviews, project organization, dashboard metrics, and profile settings.

3. Dashboard

The Dashboard shows summary counts for the signed-in user, including stored conversions and reviews.

4. Studio Conversion Workflow

The Studio is the main workspace.

4.1 Start a conversion

1. Enter a conversion name.
2. Select an output language. R is the default; Python is also available.
3. Select a source type, or keep `Auto-detect` / `Not sure`.
4. Optionally enter translation guidance or a reference URL.
5. Paste SAS code or upload a `.sas` file.
6. Attach any data files needed for validation or execution.
7. Select conversion options and click `Convert`.

A conversion name and SAS source code are required. Source-file uploads must use the `.sas` extension.

When a `.sas` file is uploaded, its content populates the source editor. If the conversion name is blank, the source filename becomes the initial conversion name.

4.2 Source types

- `Auto-detect` / `Not sure`: SAS2Py inspects the program and selects the applicable translation route.
- `SAS`: regular SAS code such as DATA steps, PROC SQL, and SAS survey procedures.
- `SAS-callable SUDAAN`: SUDAAN procedures and statements invoked from SAS, such as PROC DESCRIPT, CROSSTAB, RLOGIST, MULTLOG, NEST, SUBPOPN, and LEVELS.
- `Mixed SAS + SUDAAN`: a program that combines substantial SAS data preparation or reporting with SUDAAN analysis in the same source file.

For most users, auto-detection is the safest starting point. Select a route manually when the source is known and auto-detection needs correction.

4.3 Additional guidance and reference URLs

Additional guidance can specify methodological requirements, constraints, expected output columns, or preferred target-language approaches.

For an HTTP or HTTPS reference URL, SAS2Py attempts to fetch readable text from the page and supplies up to 8,000 characters to the model as untrusted methodological context. Fetching has a 15-second limit. Local/private-network addresses and unsupported non-text content are not read. A URL is supporting context; the SAS source remains the primary specification.

4.4 Saved-result reuse and Force regenerate

When `Force regenerate` is not selected, SAS2Py reuses the latest saved translation when all of these values match a previous conversion:

- signed-in user
- SAS source code
- output language
- additional guidance
- reference URL

Select `Force regenerate` instead of reusing the latest saved translation when a fresh model response is required. Auto-repair runs always create a new conversion rather than reusing a saved result.

4.5 Conversion output and built-in checks

After a successful conversion, the generated R or Python code appears in the output panel and is saved with the original source.

Before ordinary generated code is accepted, SAS2Py performs route-aware structural checks for known translation risks. These checks can detect patterns such as missing survey design variables, incorrect SUDAAN count logic, placeholder output values, unsupported package assumptions, and requested output artifacts that do not match the SAS program.

For the supported SUDAAN percentile pattern, the translation instructions and checks distinguish:

- `NSUM`: unweighted eligible records in the current subpopulation
- `N_ACT`: copied from `NSUM`
- `N`: separately calculated effective sample size

The checks do not run SAS and do not compare against an expected SAS result unless a reviewer independently provides one. The four repository examples are internal regression tests for SAS2Py development; they are not translated with a user's request and are not part of the user workflow.

5. SAS Analysis

Analyze SAS generates:

- a plain-language interpretation of the source
- the expected business or data output
- three to six review checks

This analysis supports review but is not an execution result or proof of statistical equivalence.

6. Input Files

Attach input data in the source section before conversion when auto-repair will run, or before manual execution. Uploaded files are exposed to Databricks through `SAS2PY_INPUT_DIR`. Generated code should resolve source data from this directory rather than reproducing local Windows paths from SAS `LIBNAME` statements.

When auto-repair is selected, SAS2Py checks filenames referenced by the SAS source before starting. If required filenames are missing or differ from the uploaded names, the app asks the user to upload the correct files. Filename matching is exact, including the extension and letter case.

Input attachments remain available when switching between R and Python. View `latest` also retains current attachments when the selected history entry has the same SAS source. Selecting or entering a different SAS source clears them.

Uploaded file contents are kept only in the current browser session and execution request. They are not stored in conversion history and cannot be restored after a page refresh or a later sign-in.

7. Automatic Run and Repair

Select `Run generated code and auto-repair runtime errors before saving` to have SAS2Py validate the generated target-language code in Databricks before the conversion is finalized.

The workflow:

1. Generates R or Python code.
2. Applies static checks for known runtime and translation risks.
3. Runs the generated code with the attached input files.
4. If the run fails, sends the error and current code back for a limited repair attempt.
5. Saves the latest code and available execution output, whether the final run passes or still needs review.

Attach the associated input files when the SAS source reads data. Auto-repair validates target-language execution; it does not execute the original SAS/SUDAAN program and usually has no known expected SAS output to compare against.

Leave auto-repair off when only a translation is needed or the required data cannot be uploaded. The code can still be run manually later.

8. Running Generated Code

R and Python code execute through Databricks Jobs. Execution results can include:

- `Run Output`

- Errors / Warnings
- exit code, duration, and timeout status
- detected packages and package-policy status
- plot previews
- downloadable output files

Users can copy, download, or paste run messages into a follow-up prompt.

If the SAS program requests one Excel workbook and no CSV files, the generated R and Python code should produce one workbook with the required sheets instead of exposing intermediate CSV files. Generated plots and other explicitly requested files are captured as downloadable outputs.

Depending on deployment settings, package allowlists or blocklists can reject a run. Large uploaded files may require Azure Blob Storage handoff when they exceed Databricks notebook-parameter limits.

9. Editing and Applying Enhancements

Users can edit generated code directly and save it to the current conversion.

`Apply enhancement` sends the current SAS source, current generated code, and the user's instruction to the refinement service. It is intended for concrete requests such as changing a plot color, improving performance, or adjusting output formatting. The app requires a real code change and retries once if the first response is unchanged; an unchanged result is reported as an error instead of being saved as a successful enhancement.

Enhancements can be applied to an entry opened with `View latest`; a new conversion is not required. The current file can be downloaded as `.R` or `.py`.

10. Reviews

Each saved conversion can receive review notes. Comments are required. A summary is optional, and an optional rating must be from 1 to 5.

11. History and Projects

History supports search by conversion name, project name, SAS source, or generated code. `View latest` opens the preferred language version for a grouped source when available.

Users can create projects, move conversions between projects, and return entries to the unassigned group. Deleting a project unassigns its conversions; it does not delete them. Deleting an individual conversion removes the entry and its stored reviews and execution records.

Input file contents are not part of history. Reattach them before a future run or auto-repair request.

12. Settings

Settings supports profile maintenance, including viewing the account email and updating the display name.

The Appearance section includes a dark-mode switch. The selected theme applies throughout SAS2Py and remains active after navigation and page refreshes.

13. Common Validation and Error Cases

- Missing conversion name or SAS source: provide both before converting.
- Incorrect input filenames: upload the exact files referenced by the source before auto-repair.
- Reference page cannot be read: continue without it or provide an accessible HTTP/HTTPS text page.
- Model timeout or rate limit: retry after the service recovers; use saved-result reuse when an identical conversion already exists.
- Package-policy rejection: revise package use or contact the deployment administrator.
- Databricks timeout or runtime error: review [Errors](#) / [Warnings](#), correct inputs or code, and run again.
- Enhancement leaves code unchanged: make the instruction concrete and retry; the unchanged version is not recorded as a successful enhancement.

14. Recommended Review Workflow

1. Confirm the source type and target language.
2. Convert the SAS source, attaching required data if auto-repair is used.
3. Review the SAS interpretation and generated code.
4. Verify survey strata, PSU, weights, subpopulation logic, formats, reference levels, and output schema.
5. Run the target code and inspect [Run Output](#), [Errors](#) / [Warnings](#), plots, and downloadable files.
6. Compare important values with trusted SAS/SUDAAN results when those results are available.
7. Apply enhancements or manual edits, then rerun affected code.
8. Record review notes and organize the conversion in History.

15. Administrator Notes

Deployment behavior depends on Azure OpenAI model access and timeout settings, Databricks Jobs configuration, package policy, storage handoff for large files, and database access. These settings should be checked when failures affect multiple users or differ between local and deployed environments.